Experiment No: 10

Aim: To build REST API using Express and Test Api using postman

DOP: **DOS:**

Theory:

1. REST API:

REST stands for Representational State Transfer. A REST API is an interface that allows applications to communicate with a server using standard HTTP methods. It is commonly used to exchange data between a client and server in JSON format.

2. Express.js:

Express.js is a lightweight and flexible web framework for Node.js. It provides features for creating web servers and REST APIs easily. Express allows developers to define routes, handle HTTP requests, process data, and send responses to clients.

3. HTTP Methods:

The REST API uses different HTTP methods for performing operations:

- GET – Retrieves data from the server.
- POST – Creates new data on the server.
- PUT – Updates existing data.
- DELETE – Removes data from the server.

4. API Endpoint:

An endpoint is a specific URL through which a client accesses an API resource. In this experiment, /api/users is used as the endpoint for user-related operations.

5. JSON:

JSON (JavaScript Object Notation) is a lightweight data format used to exchange information between the client and server. It represents data using key-value pairs and is widely used in REST APIs.
Example:

```
{
   "id": 1,
   "name": "Rahul",
   "course": "Computer Engineering"
}
```

6. Postman:

Postman is an API testing tool used to send HTTP requests to a server and examine its responses. It allows developers to test GET, POST, PUT, DELETE and other API operations without creating a separate frontend application.

7. Working of the Experiment:

The working of the experiment can be summarized as follows:

1. Node.js provides the runtime environment for executing the application.
2. Express.js is installed and used to create the REST API server.
3. Different API routes are defined for user operations.
4. The Express server listens for requests on port 3000.
5. Postman sends HTTP requests to the API endpoints.
6. Express receives and processes these requests.
7. The appropriate route is executed according to the HTTP method and URL.
8. The server sends a response, generally in JSON format, back to Postman.
9. Postman displays the response so that the API functionality can be verified.
10. Thus, the REST API can be created and tested without requiring a separate frontend application.

Program Procedure:

Requirements:

- Node.js
- Visual Studio Code
- Express.js
- Postman
- Command Prompt / PowerShell

Step 1: Verify Node.js Installation:

Open Command Prompt / PowerShell and enter: node -v

Then: npm -v

You should get the installed versions of Node.js and npm.

```
C:\Users\DELL>node -v
v22.22.2

C:\Users\DELL>npm -v
10.9.7
```

Step 2: Create a Project Folder:

Create a folder anywhere convenient, for example: REST_API_Experiment

Open this folder in VS Code.

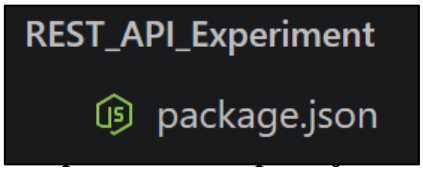
Step 3: Initialize the Node.js Project:

Open the terminal in VS Code and run: npm init -y

This creates a package.json file.

The folder structure will look like:

```
PS F:\Sem 5\Web\REST_API_Experiment> npm init -y
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```



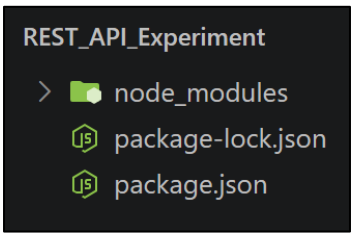
Run: npm install express
After installation, a node_modules folder and package-lock.json will be created.
Your project will now look like:

```
PS F:\Sem 5\Web\REST_API_Experiment> npm install express

added 68 packages, and audited 69 packages in 6s

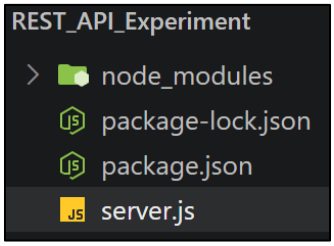
28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```



Step 5: Create the Server File and write the Express REST API Code:

1. Create a new file: server.js
2. Your folder should now contain:



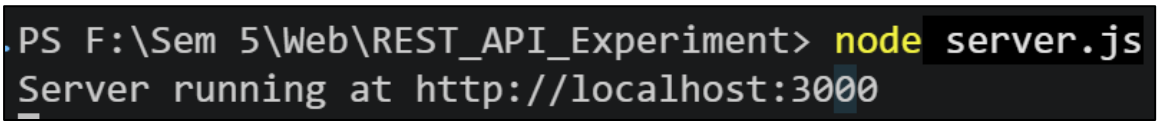
3. Open server.js and add:

```
const express = require('express');
const app = express();
const PORT = 3000;
app.use(express.json());
// GET API
app.get('/api/users', (req, res) => {
    res.json([
        { id: 1, name: 'Rahul', course: 'Computer Engineering' },
        { id: 2, name: 'Priya', course: 'Information Technology' }
    ]);
});
// GET API with ID
app.get('/api/users/:id', (req, res) => {
    const id = req.params.id;
    res.json({
        message: 'User retrieved successfully',
        userId: id
    });
});
// POST API
app.post('/api/users', (req, res) => {
    const user = req.body;
    res.status(201).json({
        message: 'User created successfully',
        user: user
```

```
    });
});
// PUT API
app.put('/api/users/:id', (req, res) => {
    const id = req.params.id;
    res.json({
        message: 'User updated successfully',
        userId: id,
        updatedData: req.body
    });
});
// DELETE API
app.delete('/api/users/:id', (req, res) => {
    const id = req.params.id;
    res.json({
        message: 'User deleted successfully',
        userId: id
    });
});
// Start server
app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

4. Save the file.

Step 6: Start the Express Server:
In the VS Code terminal, run: node server.js
You should see: Server running at http://localhost:3000
This means your Express server is running successfully.
Important: Keep this terminal running while testing the APIs.



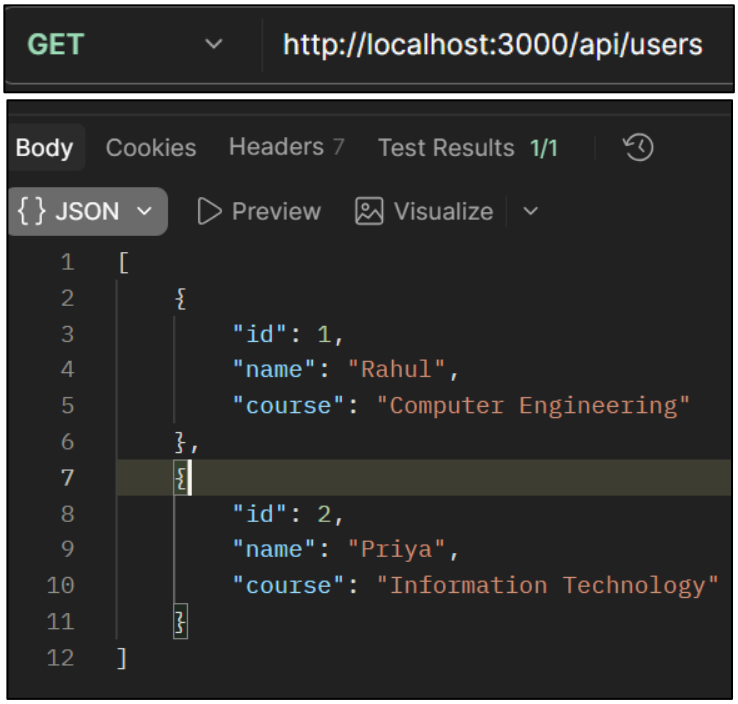
Step 7: Understand the API Endpoints:
We have created five REST API operations:

Method	Endpoint	Purpose
GET	/api/users	Retrieve all users
GET	/api/users/:id	Retrieve a specific user
POST	/api/users	Create a new user
PUT	/api/users/:id	Update a user
DELETE	/api/users/:id	Delete a user

The base URL is:http://localhost:3000
So, for example: http://localhost:3000/api/users

Step 8: Open Postman:
Open Postman.
Create a new HTTP request.
We will test each API separately.

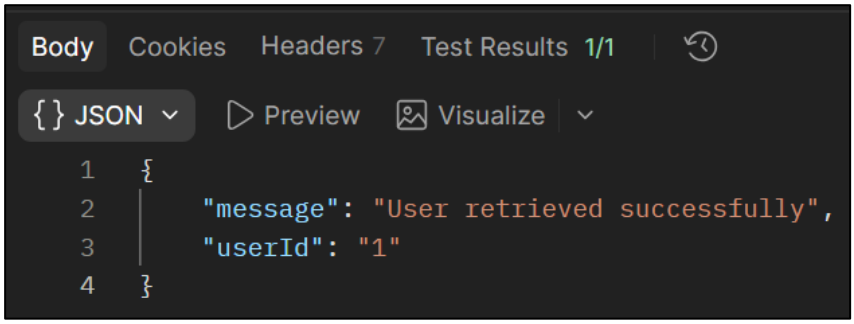
Step 9: Test GET API – Retrieve All Users:
In Postman:
Method: GET
URL: http://localhost:3000/api/users
Click: Send
You should receive:




```
[
  {
    "id": 1,
    "name": "Rahul",
    "course": "Computer Engineering"
  },
  {
    "id": 2,
    "name": "Priya",
    "course": "Information Technology"
  }
]
```

This confirms that the GET API is working.

Step 10: Test GET API with ID:
Select: GET
Enter: http://localhost:3000/api/users/1
Click Send.
Expected response:




```
{
  "message": "User retrieved successfully",
  "userId": "1"
}
```

Step 11: Test POST API – Create User:
Select: POST
Enter: http://localhost:3000/api/users
Go to: Body → raw → JSON
Enter:

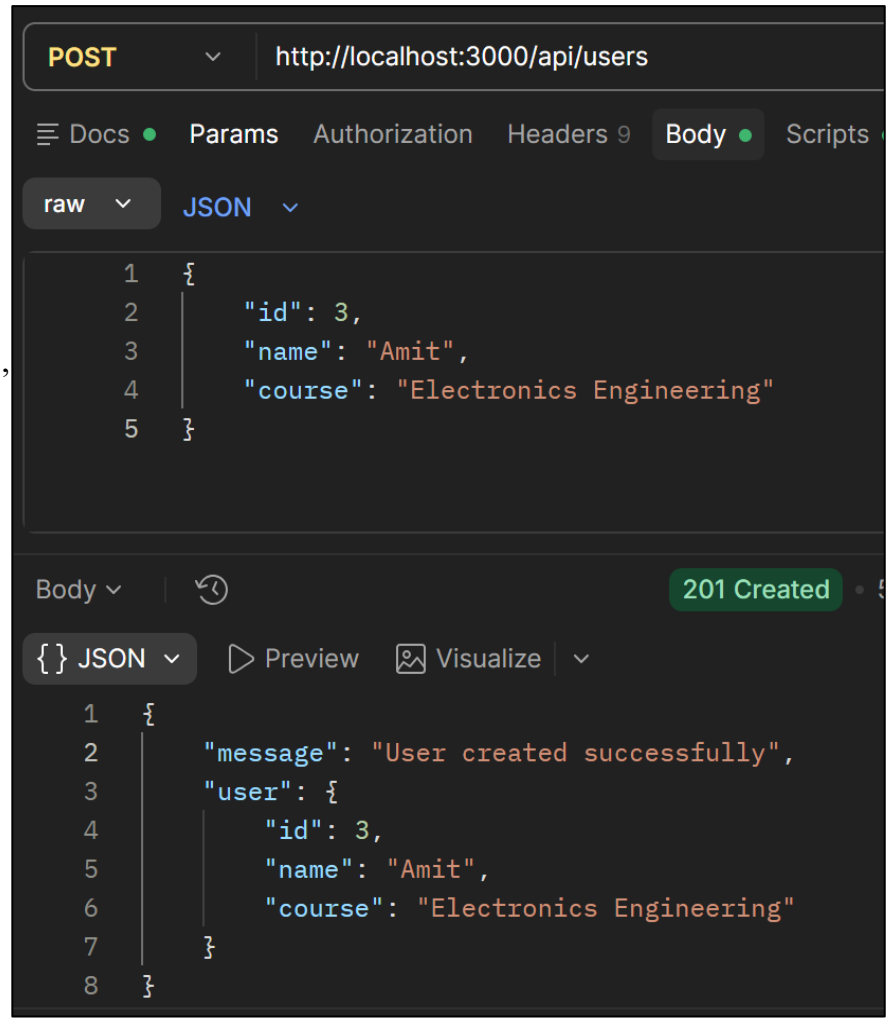
```
{
  "id": 3,
```

```
  "name": "Amit",
  "course": "Electronics Engineering"
}
```

Click Send.

Expected response:

```
{
  "message": "User created successfully",
  "user": {
    "id": 3,
    "name": "Amit",
    "course": "Electronics Engineering"
  }
}
```



Step 12: Test PUT API – Update User

Select: PUT

Enter: http://localhost:3000/api/users/1

Go to: Body → raw → JSON

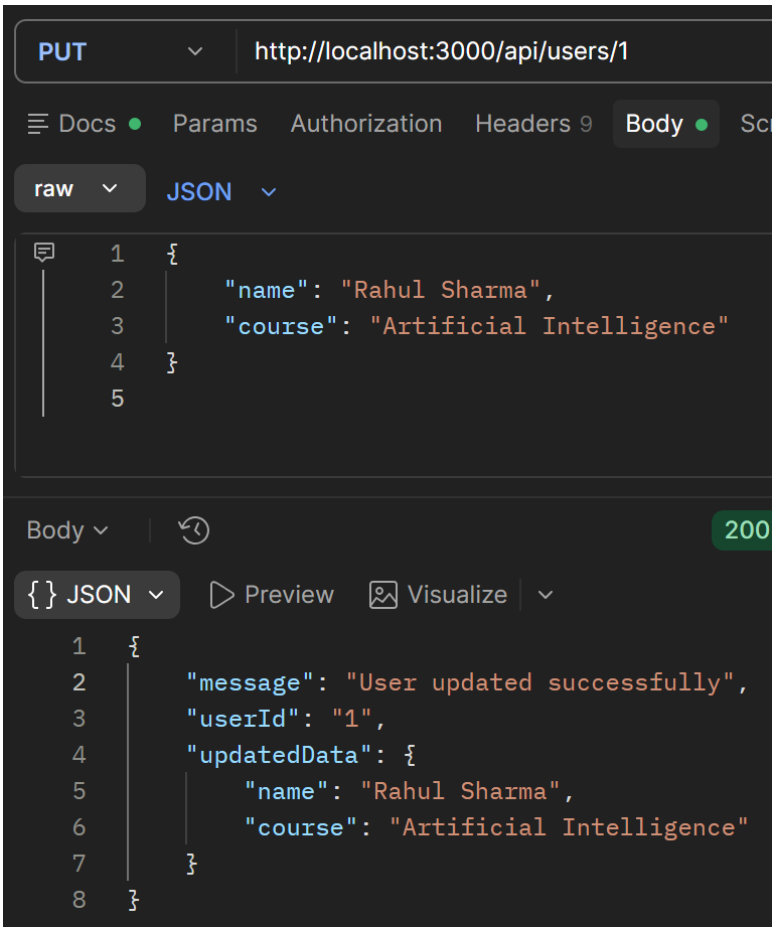
Enter:

```
{
  "name": "Rahul Sharma",
  "course": "Artificial Intelligence"
}
```

Click Send.

Expected response:

```
{
  "message": "User updated successfully",
  "userId": "1",
  "updatedData": {
    "name": "Rahul Sharma",
    "course": "Artificial Intelligence"
  }
}
```



Step 13: Test DELETE API

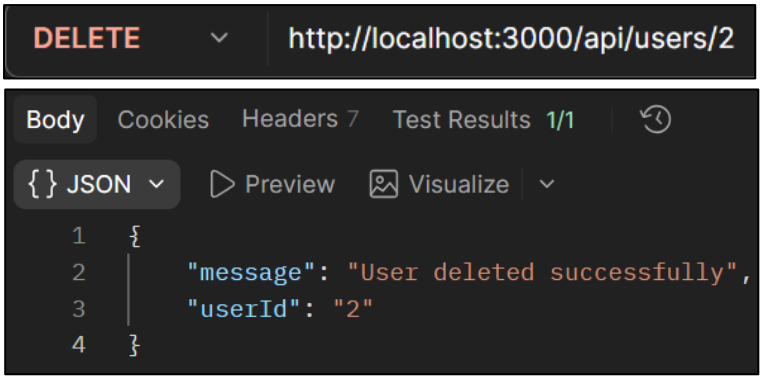
Select: DELETE

Enter: http://localhost:3000/api/users/2

Click Send.

Expected response:

```
{
  "message": "User deleted successfully",
  "userId": "2"
}
```



Step 14: Verify All APIs

At the end, we should have successfully tested:

Sr. No.	HTTP Method	API Endpoint	Result
1	GET	/api/users	Successfully retrieved users
2	GET	/api/users/1	Successfully retrieved user
3	POST	/api/users	Successfully created user
4	PUT	/api/users/1	Successfully updated user
5	DELETE	/api/users/2	Successfully deleted user

Conclusion: